National University of Computer and Emerging Sciences

Karachi Campus

Operating Systems (CS2006)

Date: October 29th 2026

Course Instructor(s)

Mr. Sandesh Kumar

Sessional-02 Exam

Total Time (Hrs): 1

Total Marks: 15

Total Questions: 3

_____ _____ _____

Roll No Section Student Signature

Do not write below this line

Attempt all the questions and do not use markers or pencils to write your answers

CLO#1: Understand and describe the concurrency (threads, multicore parallelism) and scheduling services provided by modern operating systems

Q1. [1.5+1.5+1+1+1 = 6 Marks, Estimated Time: 20 Minutes]

a) Name the most suitable scheduling approach for each system and give a one-line reason.

i) An emergency-room triage system where critically ill patients must ***always*** be served before minor cases, but nobody should wait indefinitely.
Preemptive priority scheduling with aging: priorities guarantee critical cases first, preemption handles a new critical arrival at once, aging prevents starvation of minor cases.

ii) A desktop OS that must keep the GUI responsive while a video encoder runs in the background.
MLFQ (or RR with a foreground/background MLQ): interactive tasks have short bursts and stay in high-priority short-quantum queues; the encoder sinks to lower queues.

iii) A bank's nightly batch payroll job queue where the run time of each job is known in advance and only total completion matters.
SJF (non-preemptive): burst times are known, so SJF minimizes average waiting/turnaround with no preemption overhead.

b) Classify each scenario as data or task parallelism and give a one-line reason.

i) Four threads each add one quarter of a 1-million-element array; main adds the four partial sums.
Data parallelism: same operation (addition) on different subsets of data.

ii) For the same data set, thread 1 computes the mean, thread 2 sorts a copy, and thread 3 draws a histogram.
Task parallelism: different operations on the same (or different) data.

iii) A photo editor applies the same blur filter to different tiles of an image on eight threads.
Data parallelism: identical filter on distinct image tiles.

c) A multithreaded program has a serial fraction S = 25%. Compute the speedup to N = 4 and N = ∞ cores using Amdahl's Law

$$N = 4: 1/(0.25 + 0.75/4) = 1/0.4375 = 2.29.$$
$$N \to \infty: 1/0.25 = 4.$$

d) Why is thread-local storage (TLS) needed when each thread already has local variables?
Local variables are visible only during one function invocation, TLS is visible across function invocations in that thread and behaves like a static/global unique to each thread.

e) Which web server consumes less memory: a new process or a new thread per request? Give one technical reason.
Thread per request [0.5]; threads share the process address space (code, heap, globals), a new process needs its own copy of memory/PCB [0.5].

CLO#2: Analyze and compare synchronization mechanisms by testing them against the critical-section requirements.

Q2: [3 + 3 = 6 Marks, Estimated Time: 25 Minutes]

a) A student changes Peterson's entry section to flag[i] = true; turn = i; while (flag[j] && turn == j); (i.e. turn = i instead of turn = j).

i) Show a schedule in which both processes enter the critical section (table: Step, Process, Action, turn, flag[0], flag[1]; initial turn = 0).

Step	Proc	Action	turn	flag[0]	flag[1]	Remarks
1	P0	flag[0] = true	0	T	F	
2	P0	turn = 0	0	T	F	P0 sets turn to itself
3	P0	while (flag[1] && turn == 1)	0	T	F	flag[1] is false → P0 enters CS
4	P1	flag[1] = true	0	T	T	P0 is still inside the CS
5	P1	turn = 1	1	T	T	P1 sets turn to itself
6	P1	while (flag[0] && turn == 0)	1	T	T	turn ≠ 0 → condition false → P1 enters CS

ii) State the original line and why the original lets exactly one process proceed when both want to enter.
Original: `turn = j`. A process gives away the turn: when both are interested, the one that writes turn last yields, so exactly one of the two wait conditions is false. With `turn = i` a late arrival takes the turn and passes its own loop test even though the other process is already inside.

iii) Which critical-section requirement is violated?

Mutual exclusion.

Rubric (marking scheme)	Marks
(a) Correct schedule with P0 entering first and P1 entering while P0 is inside	1.5
(b) Original line turn = j + yielding explanation	1
(c) Mutual exclusion	0.5
Total	**3**

b) Three proposed entry/exit protocols for two processes P0 and P1 (i is the current process, j = 1 − i). For each: name the requirement(s) of the critical-section problem that it violates (mutual exclusion, progress, bounded waiting) and give a concrete execution that shows the violation.

```
/* Protocol 1 */  int turn = 0;
while (turn != i) ;           /* entry */
/* critical section */
turn = j;                     /* exit  */

/* Protocol 2 */  bool flag[2] = {false, false};
flag[i] = true;  while (flag[j]) ;
/* critical section */
flag[i] = false;

/* Protocol 3 */  bool flag[2] = {false, false};
while (flag[j]) ;  flag[i] = true;
/* critical section */
flag[i] = false;
```

Protocol	Violated requirement	Execution that shows it
1 (strict alternation)	Progress (also forces a rigid order)	turn = 0. P0 enters, leaves and sets turn = 1; P1 is in its remainder section and never comes. P0 wants to enter again, but waits forever for turn = 0 even though the critical section is free: a process outside its critical section blocks another.
2 (set flag, then test)	Progress (deadlock); bounded waiting is not guaranteed. Mutual exclusion holds.	Both set their flags (flag[0] = flag[1] = true) before either tests. Both spin forever; nobody can enter although the critical section is empty.
3 (test, then set flag)	Mutual exclusion	P0 tests flag[1] (false) and is preempted before setting flag[0]; P1 tests flag[0] (false); both set their flags and both enter the critical section.

Rubric (marking scheme)	Marks
Protocol 1: correct requirement 0.25, scenario 0.75	1
Protocol 2: correct requirement 0.25, scenario 0.75	1
Protocol 3: correct requirement 0.25, scenario 0.75	1
Total	**3**

CLO#3: Design and implement multithreaded programs using the Pthread library.

Q3: Write a C program using the Pthread library that finds the maximum of an integer array of 1000 random elements using four threads. Each thread scans one quarter of the array and stores its local maximum in the global array `lmax[]`; `main()` joins the threads and computes the final maximum. [3 Marks, Estimated Time: 15 Mins]

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#define N 1000
#define T 4

int a[N];
int lmax[T];                        /* one slot per thread -> no race
*/

void *find_max(void *param) {
    int id = *(int *)param;        /* thread number 0..3 */
    int lo = id * (N / T), hi = lo + N / T;
    int m = a[lo];
    for (int i = lo + 1; i < hi; i++)
        if (a[i] > m) m = a[i];
    lmax[id] = m;                  /* store local maximum */
    pthread_exit(NULL);
}

int main(void) {
    pthread_t tid[T];
    int ids[T];                 /* separate storage for each argument
*/
    for (int i = 0; i < N; i++) a[i] = rand() % 10000;

    for (int i = 0; i < T; i++) {
        ids[i] = i;
        pthread_create(&tid[i], NULL, find_max, &ids[i]);
    }
    for (int i = 0; i < T; i++)
        pthread_join(tid[i], NULL);

    int gmax = lmax[0];
```

```
    for (int i = 1; i < T; i++)
        if (lmax[i] > gmax) gmax = lmax[i];
    printf("Maximum = %d\n", gmax);
    return 0;
}
```

Rubric (marking scheme)	**Marks**
Headers, macros, global arrays a[] and lmax[]; valid thread function signature `void *f(void *)`	0.75
Thread function: correct range (id·N/T …) and local-max scan	0.5
Thread function: stores into lmax[id] (no shared variable race) and returns/exits	0.25
Creates 4 threads in a loop with a safe argument (`&ids[i]`, not `&i`)	0.75
Join all 4 threads before using lmax[]	0.5
Computes and prints global maximum in main()	0.25
Total	**3**